Insertion operations

DSA LAB-9

5.3.26

NAME: Ninada S A

USN: 1RUA25BCA0075

1.Insertion at the beginning of a singly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
void display(struct Node* head) {
if(head == NULL) return;
struct Node* temp = head;
printf("%d -> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void insertBeginning(struct Node** head, int val) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = val;
struct Node* last = *head;
while(last->next != *head) last = last->next;
newNode->next = *head;
last->next = newNode;
*head = newNode;
}
int main() {
struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
struct Node* head = n1;
insertBeginning(&head, 5);
display(head);
return 0;
}
```

Output-

```
5 -> 10 -> 20 -> 30 -> 40 -> (Back to Head)
```

2.Insertion at the end of a singly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
void display(struct Node* head) {
if(head == NULL) return;
struct Node* temp = head;
printf("%d -> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void insertEnd(struct Node** head, int val) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = val;
struct Node* last = *head;
while(last->next != *head) last = last->next;
last->next = newNode;
newNode->next = *head;
}
int main() {
struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
struct Node* head = n1;
insertEnd(&head, 50);
display(head);
return 0;
}
```

Output-

```
10 -> 20 -> 30 -> 40 -> 50 -> (Back to Head)
```

3.Insertion at a given position of a singly circular link-list: **here pos=3**

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
void display(struct Node* head) {
if(head == NULL) return;
struct Node* temp = head;
printf("%d -> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void insertPos3(struct Node** head, int val) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = val;
struct Node* temp = *head;
for(int i=1; i<2; i++) temp = temp->next;
newNode->next = temp->next;
temp->next = newNode;
}
int main() {
struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
struct Node* head = n1;
insertPos3(&head, 25);
display(head);
return 0;
}
```

Output-

```
10 -> 20 -> 25 -> 30 -> 40 -> (Back to Head)
```

Deletion operations

DSA LAB-9
5.3.26
NAME: Ninada S A
USN: 1RUA25BCA0075

1.Deletion at the beginning of a singly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
void display(struct Node* head) {
if(head == NULL) return;
struct Node* temp = head;
printf("%d -> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void deleteBeg(struct Node** head) {
struct Node* temp = *head;
struct Node* last = *head;
while(last->next != *head) last = last->next;
last->next = (*head)->next;
*head = (*head)->next;
free(temp);
}
int main() {
struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
struct Node* head = n1;
deleteBeg(&head);
display(head);
return 0;
}
```

Output-

```
20 -> 30 -> 40 -> (Back to Head)
```

2.Deletion at the end of a singly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
void display(struct Node* head) {
if(head == NULL) return;
struct Node* temp = head;
printf("%d -> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void deleteEnd(struct Node** head) {
struct Node* temp = *head;
struct Node* prev;
while(temp->next != *head) {
prev = temp;
temp = temp->next;
}
prev->next = *head;
free(temp);
}
int main() {
struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
struct Node* head = n1;
deleteEnd(&head);
display(head);
return 0;
}
```

Output-

```
10 -> 20 -> 30 -> (Back to Head)
```

3.Deletion at a given position of a singly circular link-list:

Code-

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
void display(struct Node* head) {
if(head == NULL) return;
struct Node* temp = head;
printf("%d -> ", temp->data);
temp = temp->next;
while(temp != head) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("(Back to Head)\n");
}
void deletePos3(struct Node** head) {
struct Node* temp = *head;
struct Node* prev;
for(int i=1; i<3; i++) {
prev = temp;
temp = temp->next;
}
prev->next = temp->next;
free(temp);
}
int main() {
struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
struct Node *n4 = (struct Node*)malloc(sizeof(struct Node));
n1->data=10; n2->data=20; n3->data=30; n4->data=40;
n1->next=n2; n2->next=n3; n3->next=n4; n4->next=n1;
struct Node* head = n1;
deletePos3(&head);
display(head);
return 0;
}
```

Output-

```
10 -> 20 -> 40 -> (Back to Head)
```